Github: https://github.com/adrt1912/labpatternsIS2

Ejercicio 1 Simple Factory

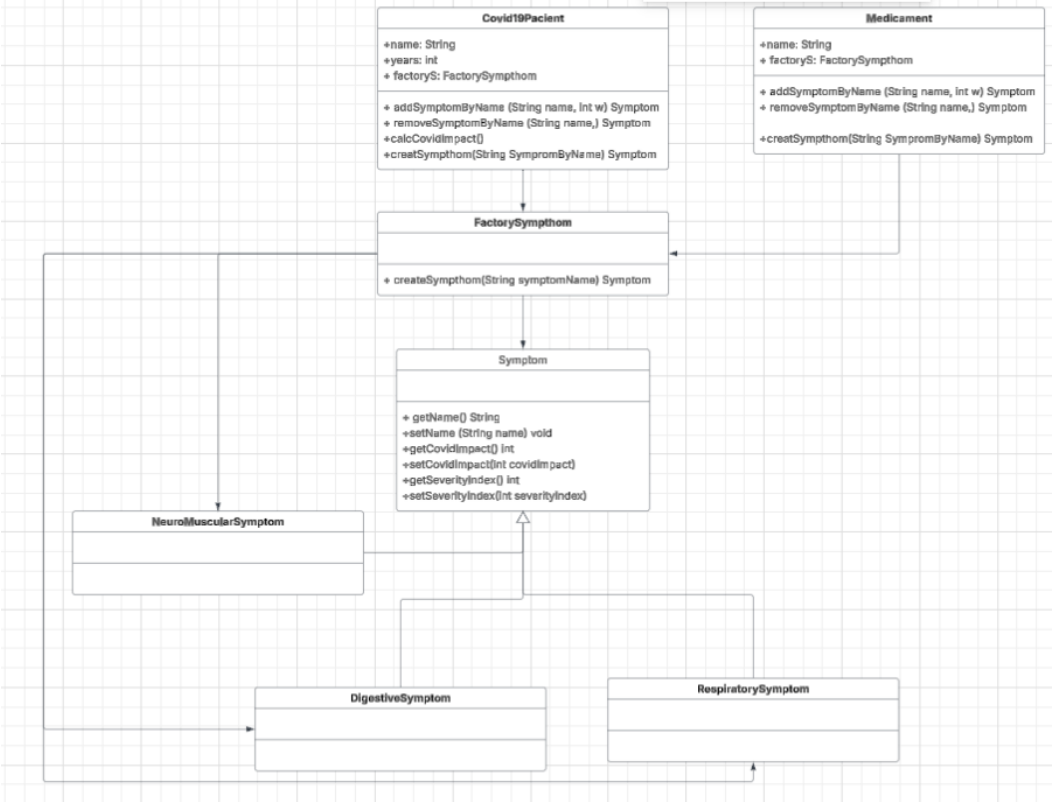
En el diseño original se incumple el principio de responsabilidad única ya que la clase Covid19Pacient se encarga tanto de gestionar los síntomas como de crearlos. Además, si se quiere añadir un nuevo tipo de síntoma (respiration) nos obliga a modificar el método createSymptom() de la clase covid19Pacient, mostrando que no esta cerrado ante modificaciones.

Para solucionar esto hay que aplicar un patrón Simple Factory en el que la nueva clase Factory solo se encargara de crear un síntoma y devolverlo. De esta manera, el resto de las clases (medicament y Covid19Pacient) delegan en la fábrica el proceso de crear el objeto, y además no necesitan conocer las subclases concretas. Y si se crea un nuevo síntoma solo hay que cambiar la clase Factory, y si es un nuevo tipo se crea e implementa la clase sintom, pero ya no hace falta cambiar código en covid19pacient ni en medicament.

Para implementar mareos sirve con actualizar en la clase FactorySympthom añadiendolo

```
List<String> impact1 = Arrays.asList("nauseas", "vómitos", "congestión nasal","diarrea","hemoptisis","congestion conjuntival","mareos");
List<Double> index1 = Arrays.asList(5.0, 4.8, 3.7, 0.9, 0.8,0.8,0.8);

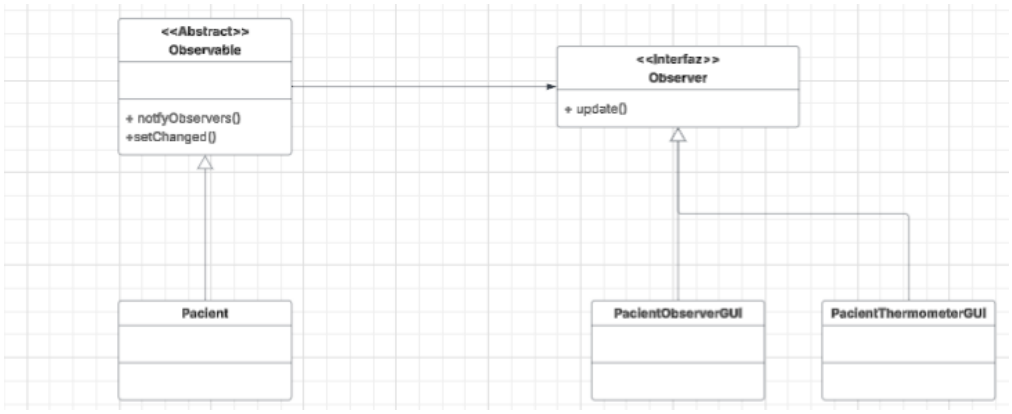
List<String> digestiveSymptom=Arrays.asList("nauseas", "vómitos","diarrea");
List<String> neuroMuscularSymptom=Arrays.asList("fiebre", "astenia", "cefalea", "mialgia","escalofrios","mareos");
```



Para el apartado c, que use el mismo síntoma ya creado tanto en medicament como en Covid19Pacient lo que he hecho es añadir un map<String,Symptom> donde se guardan los síntomas ubicados por el nombre, entonces en cada vez que se crea trata de buscarlo, si existe lo devuelve, sino creamos el nuevo síntoma y lo añadimos.

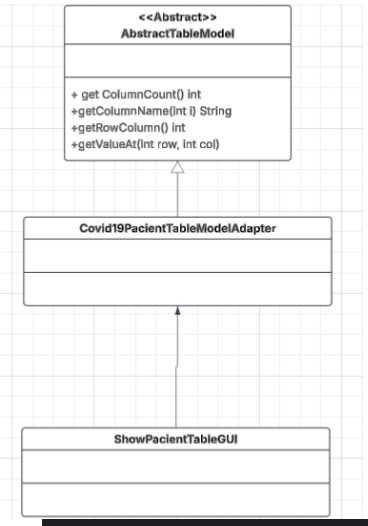
Ejercicio 2 Observer

Se ha implementado el patrón observer, para que las interfaces graficas se actualicen en caso de que el paciente cambie. Para ello los objetos que quieren ver han de implementar la interfaz observer, mientras que los que si varían extenderán la clase observable y en cada método que varíen los datos deberá ejecutar los métodos notifyObservers() y setChanged() para asi avisar los objetos.




```
    public static void main(String[] args) {
        Covid19Pacient pacient=new Covid19Pacient("aitor", 25,new FactorySympthom());
        new PacientObserverGUI (pacient);
        new PacientSymptomGUI ((Covid19Pacient)pacient);
        new PacientThermometerGUI((Covid19Pacient)pacient);
        ShowPacientTableGUI gui=new ShowPacientTableGUI(pacient);
        gui.setPreferredSize(new java.awt.Dimension(300, 200));
        gui.setVisible(true);
        Covid19Pacient pacient2=new Covid19Pacient("ane", 22,new FactorySympthom());
        new PacientObserverGUI (pacient2);
        new PacientSymptomGUI ((Covid19Pacient)pacient2);
    }
```

Ejercicio 3 Adapter



Se ha creado Covid19PacientTAbleModelAdapter para adaptar los datos del paciente al formato que necesita Jtable. El adaptador recibe un paciente y sobrescribe los métodos de AbstatTableModel para indicar la información.

Y para actualizar la tabla automáticamente sirve con añadirlo al patrón observer de antes, es decir que implemente la interfaz y añadirle el método update() para que vuelva a poner los datos en la tabla.

```
public class Covid19PacientTableModelAdapter extends AbstractTableModel {
      protected Covid19Pacient pacient;
      protected String[] columnNames =
      new String[] {"Symptom", "Weight" };
      protected Vector<Symptom> sintomas;

      public Covid19PacientTableModelAdapter(Covid19Pacient p) {
        this.pacient=p;
        sintomas=new Vector<Symptom> (p.getSymptoms());
      }

      public int getColumnCount() {
        // Challenge!
         return 2;
      }

      public String getColumnName(int i) {
        // Challenge!
          return columnNames[i];
      }

      public int getRowCount() {
        // Challenge!
          return pacient.getSymptoms().size();
      }

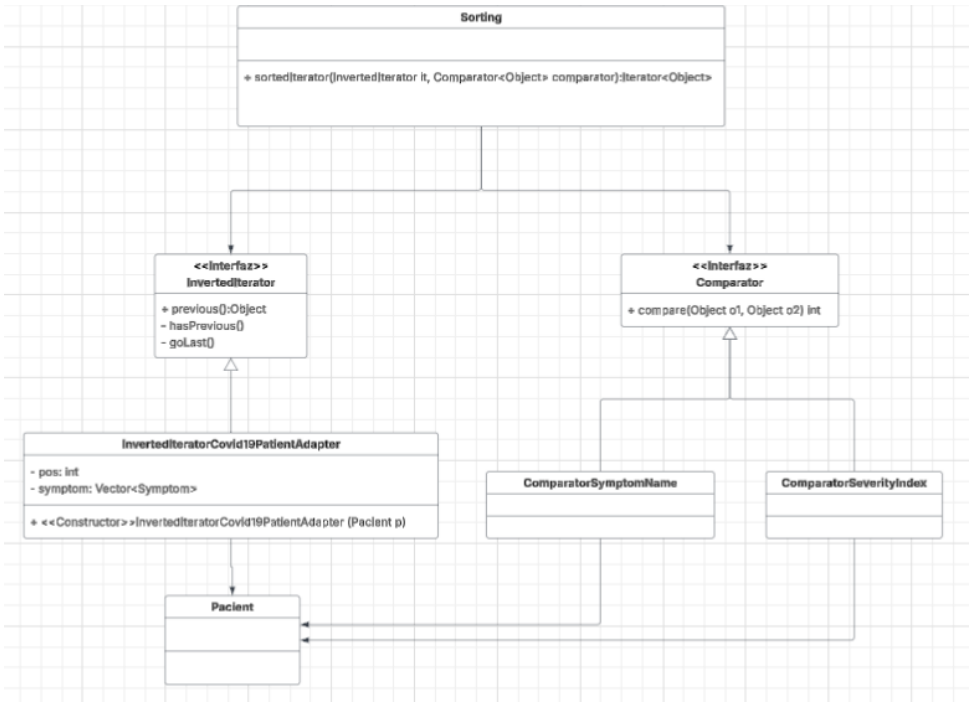
      public Object getValueAt(int row, int col) {
        // Challenge!
           sintomas=new Vector<Symptom> (pacient.getSymptoms());

          if(col==0) return sintomas.get(row).getName();
          else if (col==1) return pacient.getWeight(sintomas.get(row));
          return null;
          }
      }
```

Ejercicio 4 Patrón Iterator y Adapter

Para poder aplicar el método sortedIterator() de la clase Sorting se necesita un objeto de tipo InvertedIterator para recorrer de forma inversa los síntomas del paciente. Ya que no podemos modificar la clase Covid19Pacient se debe aplicar el patrón adapter el cual implementará la interfaz InvertedIterator y tiene una referencia a la lista de síntomas del paciente.

Además se necesita una implementación de la interfaz Comparator, para ello hemos creado dos tipos de comparadores, según el nombre y según el índice de severidad.


Sorting
+ sortedIterator(InvertedIterator it, Comparator<Object> comparator):Iterator<Object>
<<Interfaz>>
InvertedIterator
+ previous():Object
- hasPrevious()
- goLast()
<<Interfaz>>
Comparator
+ compare(Object o1, Object o2) int
InvertedIteratorCovid19PatientAdapter
- pos: int
- symptom: Vector<Symptom>
+ <<Constructor>>InvertedIteratorCovid19PatientAdapter (Pacient p)
ComparatorSymptomName
ComparatorSeverityIndex
Pacient